

Shikshana Prasarak Mandali's
SIR PARASHURAMBHAU COLLEGE
(Empowered Autonomous)
Department of Computer Science
Lab-Book
S. Y. B. Sc.
(Computer Science)
VSC Lab – Advanced Python Programming
(From Academic year 2025-26)

Name: _____
College Name: _____
Roll No.: _____ Division: _____
Academic Year: _____



Shikshana Prasarak Mandali's

SIR PARASHURAMBHAU COLLEGE

(Empowered Autonomous)

TILAK ROAD, PUNE – 411 030

Department of Computer Science

Subject - Computer Science

Certificate

*This is to certify that Mr./Ms. _____
of T.Y.B.Sc. (Computer Science) has completed _____ practicals out of
_____ in the subject **Computer Science** Semester – V/VI during the
academic year 20 - 20*

Teacher In-Charge

Head

Date:

Department of Computer Science

Examiner 1

Examiner 2

Assignment Completion Sheet

Sr. No.	Assignment Name	Marks
1	Git Programming	
2	Object-Oriented Programming (OOP)	
3	Decorators	
4	Exception handling	
5	Basic of Pandas and Numpy	
6	Data Visualization	
7	Network Programming	
8	Database Connectivity & Visualization	
	Total (Out of 80)	
	Total (Out of 20)	

Assignment 1: Git Programming

1) What is Version Control?

Version Control System (VCS) is a tool used to track changes made to files over time. It allows multiple developers to work on the same project without overwriting each other's work.

Key features:

- Records changes to files
- Allows going back to previous versions
- Supports teamwork and collaboration
- Helps maintain clean and stable project history

2) Why is Version Control Needed?

Version control is essential because:

- Prevents accidental loss of code
- Maintains different versions of a project
- Supports multiple developers working together
- Enables rollback to older stable versions
- Allows code review and tracking who made which change
- Essential for professional software development
- Most widely used version control system: Git

Most widely used remote hosting platform: GitHub

3) Installing and Configuring Git

a) Installation

- i) Windows: Download from git-scm.com
- ii) Linux: `sudo apt install git`

b) Git Configuration (run once after installation)

- i) `git config --global user.name "Your Name"`
- ii) `git config --global user.email "you@example.com"`

c) Check settings:

- i) `git config --list`

4) Basic Git Commands

a) git init

Used to create a new git repository in a folder.

git init

b) git clone

Used to copy an existing GitHub repository to your system.

git clone https://github.com/user/repo.git

c) git status

Shows current changes in the working directory.

git status

d) git add

Moves files to the staging area.

git add filename.py

To add all files:

git add .

e) git commit

Saves your changes permanently in the repository history.

git commit -m "Added new feature"

f) git log

Shows the commit history.

git log

5) Working with Remote Repositories (GitHub)

- GitHub is a platform to store git repositories online.
- Steps to work with GitHub:
 - Create an account on GitHub
 - Create a new repository
 - Link local repo to GitHub
- **Add remote repo:**
 - git remote add origin https://github.com/username/repo.git
- **Push code to GitHub:**
 - git push -u origin main
- **Pull latest changes from GitHub:**

- git pull
- **Fetch changes without merging:**
 - git fetch

6) Creating and Managing Branches (Introduction)

A branch allows you to work on new features without affecting the main codebase.

a) Create a new branch:

git branch feature1

b) Switch to the branch:

git checkout feature1

Or create + switch in one command:

git checkout -b feature1

c) View all branches:

git branch

d) Merge branch into main:

git checkout main

git merge feature1

7) Practical Examples

a) Example 1: Initialize a project

- mkdir myproject
- cd myproject
- git init

b) Example 2: Add and commit Python file

- echo "print('Hello Git')" > app.py
- git add app.py
- git commit -m "Initial commit with hello program"

c) Example 3: Push to GitHub

- git remote add origin https://github.com/username/myproject.git
- git push -u origin main

8) Summary Table

Topic	Description
Version Control	Tracks changes in code
Git	Most popular version control tool
GitHub	Remote hosting for repositories
git init	Create new repository
git add / commit	Save changes
git push / pull	Sync with GitHub
Branching	Parallel development

Note: For all the questions below, please verify the actual output. Alternatively, ask students to capture screenshots of their results, compile them into a single file, and submit it for evaluation.

SET A

- 1) Install and Configure Git. Perform the following operations
 - a) Install Git on your system.
 - b) Configure username & email:
 - c) Verify: `git config --list`
- 2) Initialize a Local Repository. Perform the following operations
 - a) Create a folder LocalGitDemo.
 - b) Add file notes.txt containing any 5 lines.
 - c) Run:
 - i) `git init`
 - ii) `git add notes.txt`
 - iii) `git commit -m "Initial commit"`
 - d) Show:
 - i) `git status`
 - ii) `git log`

3) Tracking File Changes. Perform the following operations

- a) Modify notes.txt by adding 2 more lines.
- b) View changes:
 - i) `git status`
 - ii) `git diff`
- c) Stage & commit changes:
 - i) `git add notes.txt`
 - ii) `git commit -m "Updated notes file"`

SET B

1) Working With Multiple Files. Perform the following operations

- a) Inside the same repository:
Create three files:
 - i) `app.py`
 - ii) `data.txt`
 - iii) `readme.md`
- b) Add only two files to staging:
`git add app.py readme.md`
- c) Commit changes:
`git commit -m "Added app and readme"`
- d) Verify that data.txt is untracked using:
`git status`

2) Renaming and Deleting Files Using Git. Perform the following operations

- a) Rename file using Git:
`git mv readme.md instructions.md`
- b) Delete a file using Git:
`git rm data.txt`
- c) Commit changes:
`git commit -m "Renamed file and removed data"`

SET C

1) Creating and Switching Branches. Perform the following operations

- a) Create a new branch:
git branch feature
- b) Switch to it:
git checkout feature
- c) Edit app.py:
print("Feature branch update")
- d) Commit the change:
git add app.py
git commit -m "Updated app in feature branch"

2) Viewing Commit History & Reverting. Perform the following

- a) View history:
git log --oneline
- b) Revert a previous commit (not the latest):
git revert <commit_id>

Assignment Evaluation

0: Not done ☐ 2: Late Complete ☐ 4: Complete ☐
 1: Incomplete ☐ 3: Needs improvement ☐ 5: Well Done ☐

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 1: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 2: Object-Oriented Programming (OOP)

- Python is a multi-paradigm language – it supports procedural, functional, and object-oriented programming.
- Object-Oriented Programming (OOP) is a way of structuring programs so that properties and behaviours are bundled into objects.
- The four main OOP principles are:
 - a) Encapsulation
 - b) Inheritance
 - c) Polymorphism
 - d) Abstraction
- Along with these, concepts like classes, objects, constructors, instance/class/static methods, method overriding, and operator overloading are also key.

1) Classes and Objects

- A class is a blueprint for creating objects.
- An object is an instance of a class.
- **Example:**

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand    # instance variable
        self.model = model

    def display(self):
        print(f"Brand: {self.brand}, Model: {self.model}")

# Creating objects
c1 = Car("Toyota", "Innova")
c2 = Car("Honda", "City")

# display data
c1.display()
c2.display()
```

2) Constructors (__init__) and self

- __init__() is a special method (constructor) used to initialize objects.
- self represents the current object.

- **Example:**

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

    def show(self):
        print("Name:", self.name, "| Roll No:", self.roll)

s1 = Student("Rahul", 101)
s2 = Student("Priya", 102)

s1.show()
s2.show()
```

3) Instance, Class, and Static Methods

- Instance methods: Work with object attributes (use self).
- Class methods: Work with class variables (use @classmethod and cls).
- Static methods: Independent methods that don't access class/object variables (use @staticmethod).

- **Example:**

```
class Employee:
    company = "ABC Corp" # class variable
    count = 0            # to track total employees

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.count += 1
```

```

def show(self): # instance method
    print(f'Employee: {self.name}, Salary: {self.salary}')

    @classmethod
    def total_employees(cls): # class method
        print("Total Employees:", cls.count)

    @staticmethod
    def is_eligible(age): # static method
        return age >= 18

e1 = Employee("Anil", 50000)
e2 = Employee("Sunita", 60000)

e1.show()
e2.show()
Employee.total_employees()
print("Age 16 eligible?", Employee.is_eligible(16))

```

4) Inheritance

- Inheritance allows a class (child) to acquire properties and methods of another class (parent).
- Supports single, multiple, and multilevel inheritance.
- **Example:**

```

class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self): # overriding
        print("Dog says Woof!")

```

```
class Cat(Animal):
    def speak(self):
        print("Cat says Meow!")
```

```
d = Dog()
c = Cat()
d.speak()
c.speak()
```

Output:

Dog says Woof!
Cat says Meow!

5) Encapsulation (Data Hiding)

- Encapsulation means restricting direct access to variables.
- In Python, private members use double underscore (__).
- **Example:**

```
class BankAccount:
    def __init__(self, acc_no, balance):
        self.acc_no = acc_no
        self.__balance = balance # private variable
```

```
    def deposit(self, amount):
        self.__balance += amount
```

```
    def get_balance(self):
        return self.__balance
```

```
acc = BankAccount(101, 5000)
acc.deposit(2000)
print("Balance:", acc.get_balance())
# print(acc.__balance) # ❌ Error: private variable
```

6) Abstraction

- Abstraction is hiding implementation details and showing only functionality.
- In Python, we use the abc module (Abstract Base Class).

- **Example:**

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```

```
class Circle(Shape):  
    def __init__(self, radius):  
        self.radius = radius  
  
    def area(self):  
        return 3.14 * self.radius * self.radius
```

```
c = Circle(5)  
print("Area of Circle:", c.area())
```

7) Polymorphism

- Polymorphism = same function name, different implementations.
- Achieved using method overriding or duck typing.

- **Example (Duck Typing):**

```
class Dog:  
    def sound(self):  
        return "Woof!"
```

```
class Cat:  
    def sound(self):  
        return "Meow!"
```

```
def make_sound(animal):
    print(animal.sound())

make_sound(Dog())
make_sound(Cat())
```

8) Operator Overloading

- In Python, we can overload operators by defining special methods like `__add__`, `__sub__`, etc.

- **Example:**

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other): # overloading +
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f'({self.x}, {self.y})'

v1 = Vector(2, 3)
v2 = Vector(4, 5)
print(v1 + v2) # (6, 8)
```

SET A

- 1) Write a Python program to define a class Student with attributes name, roll_number, and marks. Define a method display_details() to print the student's information. Create at least two student objects and display their details. Accept input from user.
- 2) Write a Python program to define a class Book that uses the `__init__` constructor to initialize book details: title, author, and price. Include a method show_book_info() to display all book details. Accept input from user.
- 3) Write a Python program to define a class Calculator that includes:

- A constructor to initialize two numbers.
- Instance methods add(), subtract(), multiply(), and divide().
- Demonstrate these methods using an object. Accept input from user.

SET B

1. Design an Employee Management System where Employee is a base class and Developer, Tester, and Manager are derived classes.
2. Create a class ATM with a private data member __pin.
Provide a public method to validate the PIN entered by the user and display whether access is granted or denied.
3. Create a base class User and derive two classes Customer and DeliveryPerson.
Demonstrate how derived classes inherit the properties of the base class.
4. Create a base class University, derive a class College from it, and further derive a class Student. Demonstrate how a Student object can access the methods of all parent classes

SET C

1. Design a database system where:
The database connection logic is abstract
The database connection must be closed when the object is destroyed
Write a Python program using abstract class and destructor.
Explain how resource management is achieved.
2. An organization has different types of employees such as Manager and Developer.
Each employee has a different salary calculation method.
Design a Python program to demonstrate method overriding.
Explain how runtime polymorphism is achieved.

Assignment Evaluation

0: Not done

2: Late Complete

4: Complete

1: Incomplete

3: Needs improvement

5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 2: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 2: Decorators

What is a Decorator?

A decorator is a special function that wraps another function and adds additional functionality to it.

In Python, functions are first-class objects, which means:

- Functions can be passed as arguments
- Functions can return other functions
- Functions can be assigned to variables

A decorator uses these properties to extend function behavior.

General Syntax

```
@decorator_name
```

```
def function_name():
```

```
    pass
```

This is equivalent to:

```
function_name = decorator_name(function_name)
```

Example of a Simple Decorator

```
def my_decorator(func):
```

```
    def wrapper():
```

```
        print("Function execution started")
```

```
        func()
```

```
        print("Function execution finished")
```

```
    return wrapper
```

```
@my_decorator
```

```
def display():
```

```
    print("Hello Python")
```

```
display()
```

Output

Function execution started

Hello Python

Function execution finished

Explanation

1. display() function is passed to my_decorator.
2. my_decorator returns the wrapper function.
3. wrapper adds extra behavior before and after display().

2. Static Method (@staticmethod)

Definition

A static method is a method that:

- Belongs to a class
- Does not access instance variables
- Does not access class variables

It behaves like a normal function placed inside a class for logical grouping.

Syntax

```
class ClassName:
```

```
    @staticmethod
```

```
    def method_name(parameters):
```

```
        statements
```

Example of Static Method

```
class Calculator:
```

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

```
result = Calculator.add(10, 5)
```

```
print(result)
```

Output

```
15
```

Explanation

- add() does not depend on object data or class data.
- It simply performs a calculation.
- Therefore it is declared as a static method.

Calling Static Method

Static methods can be called using:

Using Class Name

```
Calculator.add(5, 3)
```

Using Object

```
obj = Calculator()
```

```
obj.add(5, 3)
```

When to Use Static Method

Use static methods when:

- The method does not use self or cls
- The method performs utility operations

Example:

- Mathematical operations
- Validation functions
- Helper functions

3. Class Method (@classmethod)

Definition

A class method is a method that works with the class itself instead of instance objects.

It takes cls as the first parameter, which represents the class.

Syntax

```
class ClassName:
```

```
@classmethod
```

```
def method_name(cls, parameters):
```

```
    statements
```

Example of Class Method

```
class Student:
```

```
    college = "ABC College"
```

```
@classmethod
```

```
def change_college(cls, new_name):
```

```
    cls.college = new_name
```

Calling Class Method

```
Student.change_college("XYZ College")  
print(Student.college)
```

Output

XYZ College

Explanation

- college is a class variable
- change_college() modifies the class variable
- cls refers to the class itself

When to Use Class Method

Use class methods when:

- You need to access or modify class variables
- The method works with the class instead of objects

4. Property (@property)

Definition

The @property decorator is used to convert a method into an attribute.

This allows a method to be accessed without parentheses.

Normal Method Example

```
class Student:
```

```
    def percentage(self):
```

```
        return 90
```

Call:

```
s.percentage()
```

Using @property

```
class Student:
```

```
    def __init__(self, phy, chem, math):
```

```
        self.phy = phy
```

```
        self.chem = chem
```

```
        self.math = math
```

```
    @property
```

```
def percentage(self):  
    return (self.phy + self.chem + self.math) / 3
```

Usage:

```
s = Student(80, 90, 100)
```

```
print(s.percentage)
```

Output

90.0

Why Property is Useful

Suppose we change marks later:

```
s.math = 70
```

```
print(s.percentage)
```

Output automatically updates:

80.0

The percentage recalculates automatically.

SET A

1) Write a Python Program to define two classes:

- Person with attributes name and age and a method display().
- Student that inherits from Person and adds roll_number and marks.
- Demonstrate how inheritance works by creating a Student object.
- Use '@property' for base and derived class
- Accept input from user.

2) Write a Python program with classes Vehicle and Car where:

- Vehicle has a method start() that prints "Starting vehicle".
- Car that inherits from Vehicle and overrides the start() method to print "Starting car".
- Create objects of both classes and call start().
- Accept input from user.

3) Write a Python program to define a base class Shape with a method area(). Create two derived classes Rectangle and Triangle that override area() method to calculate

the respective areas. Demonstrate polymorphism by calling area() on different objects.
Accept input from user.

SET B

- 1) Write a Python program to demonstrate the concept of abstraction
 - Create an abstract class Appliance with an abstract method operate().
 - Create two subclasses WashingMachine and Microwave that inherit from Appliance and implement the operate() method.
 - Create objects of both subclasses and call their operate() method to display appropriate messages.
 - Accept input from user.
- 2) Write a Python program to demonstrate multiple inheritance and method resolution order. All attributes from all classes should be declared as private. Create a property for all attributes in each class.
 - Create a class Teacher with attributes (name, subject) and a method display_info() to show details.
 - Create another class Researcher with attributes (field, publications) and a method display_info() to show details.
 - Create a class Professor that inherits from both Teacher and Researcher.
 - Create an object of Professor and demonstrate how Python resolves method calls using Method Resolution Order (MRO).
 - Accept input from user.

SET C

Write a Python program to demonstrate abstraction.

- Create an abstract class Payment with an abstract method make_payment(amount)
- Implement three subclasses:
 - a) CreditCardPayment
 - b) UPIPayment
 - c) NetBankingPayment

Each subclass should provide its own implementation of the make_payment() method to process payments.

- Create objects of each subclass and call the make_payment() method to show payment processing.

- Use '@property'
- Accept input from user.

Assignment Evaluation

0: Not done

2: Late Complete

4: Complete

1: Incomplete

3: Needs improvement

5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 3: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 4: Exception handling**1) Introduction – Exception Handling**

- Errors are common in programs. Some errors occur due to mistakes in code (syntax errors), while others occur during execution (runtime errors).
- Exception Handling in Python allows us to detect, handle, and recover from runtime errors gracefully, instead of crashing the program.

2) Types of Errors

- **Syntax Errors**

- Caused when the rules of Python syntax are not followed.
- These are detected before execution.

- **Example:**

```
# Missing colon → SyntaxError
```

```
if True
```

```
    print("Hello")
```

- Runtime Errors (Exceptions)

- Occur during execution of the program.
- Examples: division by zero, file not found, invalid type conversion.

- Example:

```
# Division by zero → Runtime Error
```

```
num = 10 / 0
```

3) Exception Handling Blocks

- Python provides specific keywords for handling exceptions:
 - try → Code that may cause an error.
 - except → Handles the exception if one occurs.
 - else → Executes if no exception occurs.
 - finally → Always executes (useful for cleanup like closing files).

- **Example:**

```
try:
```

```
    num1 = int(input("Enter numerator: "))
```

```
    num2 = int(input("Enter denominator: "))
```

```
    result = num1 / num2
```

```
except ZeroDivisionError:
```

```

    print("Error: Division by zero is not allowed.")
except ValueError:
    print("Error: Invalid input, please enter numbers only.")
else:
    print("Result =", result)
finally:
    print("Program execution completed.")

```

4) Raising Exceptions

We can manually raise exceptions using the raise keyword.

Example:

```

age = int(input("Enter age: "))
if age < 0:
    raise ValueError("Age cannot be negative.")
else:
    print("Valid Age:", age)

```

5) Creating Custom Exception Classes

Python allows us to create user-defined exceptions by inheriting from the Exception class.

Example:

```

# Custom Exception
class InvalidMarksError(Exception):
    def __init__(self, marks):
        super().__init__(f"Invalid marks: {marks}. Marks must be between 0 and 100.")

# Usage
marks = int(input("Enter marks: "))
try:
    if not (0 <= marks <= 100):
        raise InvalidMarksError(marks)
    else:
        print("Marks are valid.")
except InvalidMarksError as e:
    print("Error:", e)

```

SET A

1. Write a Python program to handle division by zero using try and except.
2. Write a program that asks the user to enter a number and handles ValueError if the input is not an integer.
3. Create a program that reads a file (sample.txt) and uses try-except to handle FileNotFoundError.
4. Write a Python program that converts user input into an integer and catches both ValueError and TypeError.
5. Write a Python program to catch multiple exceptions (ZeroDivisionError, ValueError) in a single try block.
6. Create a program with a function that raises a custom exception NegativeNumberError if a negative number is passed.
7. Write a Python program using else and finally blocks along with try-except. Explain when each block executes.

SET B

1. Write a Python program to handle exceptions while accessing a dictionary key. Use KeyError to provide a default value.
2. Write a program that takes a list of numbers and calculates the reciprocal of each number. Handle ZeroDivisionError for zeros and TypeError for non-numeric elements.
3. Write a Python program that uses nested try-except blocks to handle exceptions at multiple levels.
4. Create a program that reads integers from a file, handles errors during file reading, and skips invalid data with proper logging.

SET C

1. Write a Python program that raises a custom exception when a string length exceeds a limit, and handle it gracefully in the main program.
2. Write a program to simulate ATM withdrawal: Raise error if balance insufficient. Handle error properly.
3. Write a program that: Accepts a number. Raises a ValueError if number is negative. Handles the exception. Uses else and finally.

4. Create a user-defined exception called InvalidAge. Raise it if age < 18. Handle it properly.

Assignment Evaluation

0: Not done

2: Late Complete

4: Complete

1: Incomplete

3: Needs improvement

5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 4: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 5: Basics of Pandas and Numpy**INTRODUCTION TO NUMPY****Objective:**

To understand NumPy library and its usage in numerical computing.

Theory:

NumPy (Numerical Python) is a Python package used for high-performance mathematical operations, array handling, and scientific computing.

Key Features:

- Fast operations on arrays
- Multi-dimensional array support
- Built-in mathematical functions
- Used in ML, Data Science, AI

Basic Commands:

import numpy as np

- **NUMPY ARRAY – CREATION & OPERATIONS**

Objective:

To study array creation methods, attributes, and simple operations.

Programs:**Creating Arrays**

```
a = np.array([1, 2, 3])
```

```
b = np.array([[1, 2], [3, 4]])
```

Special Arrays

```
np.zeros((3,3))
```

```
np.ones((2,2))
```

```
np.eye(3)
```

```
np.arange(0, 10, 2)
```

```
np.linspace(1, 5, 5)
```

Basic Operations

```
a + 2
```

```
a * 3
```

```
np.mean(a)
```

```
np.sum(a)
```

np.sqrt(a)

- **INDEXING, SLICING & RESHAPING**

Objective:

To understand array manipulation using indexing and slicing.

Examples:

a[0], a[-1]

b[:, 1]

a.reshape(3, 2)

a.flatten()

- **MATHEMATICAL & STATISTICAL FUNCTIONS**

Objective:

To use NumPy mathematical and statistical functions.

Examples:

np.min(a)

np.max(a)

np.std(a)

np.var(a)

np.sin(a)

np.cos(a)

INTRODUCTION TO PANDAS

Objective:

To understand Pandas library for data manipulation.

Theory:

Pandas provides data structures like Series and DataFrame for handling tabular data.

Importing Pandas

import pandas as pd

- **SERIES – CREATION & OPERATIONS**

Examples:

s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])

s.sum()

s.mean()

s.max()

- **DATAFRAME – CREATION & CLEANING**

Examples:

```
df = pd.DataFrame({  
    'Name': ['A', 'B', 'C'],  
    'Marks': [80, 90, 70]  
})  
df.dropna()  
df.fillna(0)  
df.rename(columns={'Marks': 'Score'}, inplace=True)
```

- **DATA SELECTION & FILTERING**

Examples:

```
df['Name']  
df[['Name', 'Marks']]  
df.iloc[0:2]  
df[df['Marks'] > 75]
```

- **FILE HANDLING – CSV & EXCEL**

Importing Files

```
df = pd.read_csv('file.csv')  
df = pd.read_excel('file.xlsx')
```

Exporting Files

```
df.to_csv('output.csv', index=False)  
df.to_excel('output.xlsx', index=False)
```

SET A

NumPy

1. Create a NumPy array from 1 to 10 and print only even numbers.
2. Create a 3×3 ones matrix and calculate its sum.

Pandas

3. Create a Series of first 5 prime numbers and find its mean.
4. Create a DataFrame with Name & Age and print only the Age column.

SET B

NumPy

1. Create a 4×4 matrix and replace values > 10 with 0.
2. Generate 20 random numbers and find mean, median & standard deviation.

Pandas

3. Load a CSV and display rows where Marks > 80.
4. Group DataFrame by “Class” and find average marks.

SET C

1. Multiply two matrices without using np.dot().
2. Clean a dataset:
 1. Remove duplicates
 2. Fill missing values with mean
 3. Sort by two columns (ASC + DESC)

Assignment Evaluation

0: Not done ☐ 2: Late Complete ☐ 4: Complete ☐
1: Incomplete ☐ 3: Needs improvement ☐ 5: Well Done ☐

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 5: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 6: Data Visualization

Introduction to Data Visualization

Data Visualization is the process of representing data in graphical or visual form such as charts, graphs, and plots. It helps users understand patterns, trends, correlations, and insights in data more easily than raw numbers.

Python provides several powerful libraries for data visualization.

Advantages of Data Visualization

- Makes complex data easy to understand
- Helps identify patterns and trends
- Supports decision making
- Useful in data analysis and machine learning

2. Popular Python Libraries for Data Visualization

1. Matplotlib

- Most commonly used library
- Provides basic plots like line, bar, scatter, histogram

2. Seaborn

- Built on top of Matplotlib
- Used for statistical visualization
- Produces more attractive graphs

3. Plotly

- Used for interactive visualization

4. Pandas Visualization

- Pandas also supports basic plotting directly from dataframes

3. Matplotlib Library

Matplotlib is the core Python library for data visualization.

Installation

```
pip install matplotlib
```

Importing Matplotlib

```
import matplotlib.pyplot as plt
```

4. Line Plot

A line plot shows the relationship between two variables.

Example

```
import matplotlib.pyplot as plt
```

```
x = [1,2,3,4,5]
```

```
y = [10,20,25,30,40]
```

```
plt.plot(x,y)
```

```
plt.title("Line Graph Example")
```

```
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
```

```
plt.show()
```

Explanation

- plot() → creates line graph
- title() → graph title
- xlabel() → x-axis label
- ylabel() → y-axis label
- show() → displays graph

Uses

- Showing trends over time
- Temperature changes
- Stock market data

5. Bar Chart

A bar chart compares different categories.

Example

```
import matplotlib.pyplot as plt
```

```
subjects = ["Math","Physics","Chemistry","Biology"]
```

```
marks = [85,90,78,88]
```

```
plt.bar(subjects,marks)
```

```
plt.title("Student Marks")
```

```
plt.xlabel("Subjects")
```

```
plt.ylabel("Marks")
```

```
plt.show()
```

Uses

- Comparing categories
- Sales comparison
- Student performance

6. Pie Chart

A pie chart represents data in percentage form.

Example

```
import matplotlib.pyplot as plt
```

```
labels = ["Python","Java","C++","JavaScript"]
```

```
sizes = [40,25,20,15]
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
```

```
plt.title("Programming Language Popularity")
```

```
plt.show()
```

Explanation

- labels → category names
- sizes → values
- autopct → shows percentage

7. Histogram

A histogram shows the distribution of numerical data.

Example

```
import matplotlib.pyplot as plt
```

```
marks = [55,60,65,70,75,80,85,90,95,100]
```

```
plt.hist(marks)
```

```
plt.title("Marks Distribution")
```

```
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```

Uses

- Data distribution
- Exam score analysis
- Age distribution

8. Scatter Plot

A scatter plot shows the relationship between two variables.

Example

```
import matplotlib.pyplot as plt
```

```
x = [1,2,3,4,5]
y = [10,15,20,25,30]
```

```
plt.scatter(x,y)
plt.title("Scatter Plot Example")
plt.xlabel("X values")
plt.ylabel("Y values")
plt.show()
```

Uses

- Finding correlation between variables
- Machine learning analysis

9. Seaborn Library

Seaborn is used for advanced and statistical data visualization.

Installation

```
pip install seaborn
```

Import

```
import seaborn as sns
```

10. Seaborn Example

Dataset Example

```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")

sns.scatterplot(x="total_bill", y="tip", data=tips)

plt.show()
```

Explanation

- `load_dataset()` → loads sample dataset
- `scatterplot()` → creates scatter plot

11. Pandas Visualization

Pandas can plot graphs directly.

Example

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Year": [2019, 2020, 2021, 2022],
    "Sales": [100, 150, 200, 250]
}

df = pd.DataFrame(data)

df.plot(x="Year", y="Sales", kind="line")

plt.show()
```

12. Types of Visualization in Python

Type	Purpose
Line Plot	Shows trends over time
Bar Chart	Compares categories
Pie Chart	Shows percentage distribution
Histogram	Shows frequency distribution
Scatter Plot	Shows relationship between variables
Box Plot	Shows statistical distribution

13. Real-Life Applications

Data visualization is used in:

- Business analytics
- Stock market analysis
- Machine learning
- Healthcare analytics
- Marketing analysis
- Scientific research

Example:

- Sales trends
- Customer behavior
- Weather forecasting
- Population analysis

SET A

1) Write a Python program using **matplotlib** to plot a simple line chart of the following data:

- $x = [1, 2, 3, 4, 5]$
- $y = [10, 20, 15, 25, 30]$

Add:

- Title
- X and Y labels
- Legend

- 2) Write a program to draw a bar chart showing the number of students enrolled in subjects:

Subject	Students
Python	50
Java	40
C++	30
JS	45

Add a title, X and Y-axis label and legend

- 3) Write a python program to create a pie chart for market share of mobile brands:

Samsung (35%), Apple (25%), Xiaomi (20%), Others (20%)

Show labels and percentage on the chart.

- 4) Write a Python program to plot a histogram of student marks:

[55, 60, 65, 70, 72, 75, 80, 82, 85, 90, 92]

SET B

- 1) Write a Python program to plot a scatter chart using the following data:

Height (cm)	Weight (kg)
150	55
160	60
165	62
170	68
175	75
180	80

Customize the plot by adding:

- Marker shape
- Title
- Labels

- 2) Write a Python program to plot a line chart with the following data:

x = [1, 2, 3, 4, 5]

y = [5, 10, 15, 8, 20]

Customize it using:

- Red colored line
- Dashed style (--)
- Circular markers (o)
- Add legend
- Add X & Y axis

3) Create a Python program using seaborn to generate a boxplot for the marks of three subjects:

Maths: [78, 82, 85, 90, 88, 76]

Science: [70, 75, 80, 85, 82, 78]

English: [65, 70, 72, 75, 78, 80]

Use:

- a) DataFrame
- b) Labels for subjects
- c) Chart title

SET C

1. Import dataset "student_marks.csv".

- a. Display column names and shape of dataset.
- b. Display random 5 rows from the dataset.
- c. Display the maximum and minimum marks obtained by students.
- d. Plot a **histogram of student marks**.

2. Import dataset "iris.csv".

- a. Display the first 5 rows of the dataset.
- b. Display the number of records of each species.
- c. Create a scatter plot between sepal length and petal length.
- d. Create a box plot for sepal width.

3. Write a Python program to read "**StudentsPerformance.csv**" dataset and perform the following:

- a. Display top 10 rows.
- b. Check missing values in the dataset.
- c. Replace missing values with mean value.
- d. Create a bar chart showing average marks of students in different subjects.

Assignment Evaluation

0: Not done

2: Late Complete

4: Complete

1: Incomplete

3: Needs improvement

5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 6: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 7: Network Programming

1) Introduction to Socket Programming

A **socket** is an endpoint in a network communication. In Python, the socket module provides functions and classes to create networked applications.

Sockets allow two programs to communicate:

- A **server** waits for and accepts connections.
- A **client** initiates the connection to the server.

Communication can be:

- **TCP (Transmission Control Protocol)** – connection-oriented, reliable
- **UDP (User Datagram Protocol)** – connectionless, faster but no guarantee

For client-server applications, we commonly use **TCP sockets**.

2) Server–Client Model Overview

A TCP communication involves the following steps:

Server

- a) Create a socket → `socket.socket()`
- b) Bind it to an address and port → `bind()`
- c) Listen for incoming connections → `listen()`
- d) Accept a connection → `accept()`
- e) Communicate using `send()` and `recv()`
- f) Close the socket

Client

- a) Create a socket
- b) Connect to server → `connect()`
- c) Send/Receive messages
- d) Close socket

3) Important Socket Functions

a) `socket.connect()` (Client side)

- Used by the client to connect to a server.
`client_socket.connect(('localhost', 8080))`

b) `socket.send()`

- Used to send data (bytes) to the connected socket.
`client_socket.send("Hello Server".encode())`

c) socket.recv()

- Used to receive data from a socket.
data = client_socket.recv(1024) # up to 1024 bytes
print(data.decode())

d) socket.bind() (Server side)

- Assigns an IP and port to the server socket.
server_socket.bind(('localhost', 8080))

e) socket.listen()

- Puts the server into passive listening mode.
server_socket.listen(5) # queue up to 5 clients

f) socket.accept()

- Accepts an incoming client connection.
client_conn, client_addr = server_socket.accept()
print("Connected with", client_addr)

4) Simple Client–Server Example

a) Server Program (server.py)

```
import socket

# Step 1: Create socket (IPv4, TCP)
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Step 2: Bind to IP & port
server_socket.bind(('localhost', 5000))

# Step 3: Listen for connections
server_socket.listen(1)
print("Server is listening on port 5000...")

# Step 4: Accept connection
conn, addr = server_socket.accept()
print("Connected by", addr)
```

```
# Step 5: Receive message
data = conn.recv(1024).decode()
print("Client says:", data)

# Step 6: Send reply
conn.send("Hello from Server!".encode())

# Step 7: Close connection
conn.close()
server_socket.close()
```

b) Client Program (client.py)

```
import socket

# Step 1: Create socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Step 2: Connect to server
client_socket.connect(('localhost', 5000))

# Step 3: Send message
client_socket.send("Hello Server!".encode())

# Step 4: Receive reply
data = client_socket.recv(1024).decode()
print("Server says:", data)

# Step 5: Close
client_socket.close()
```

5) How to Run This Program

- a) Save the two programs as server.py and client.py.
- b) First run the **server**:
`python server.py`
- c) Then run the **client**:
`python client.py`
- d) Observe the message exchange.

SET A

- 1) Write a Python program to write a TCP server and TCP client with port number 50000.
Connect client to server
 - a) Display a message “Server started” at server side
 - b) Display a message “Connected to server” at client side
- 2) Write a Python program to write a TCP server and TCP client with port number 51000.
Connect client to server
 - a) Client should send message to server
 - b) Display message on server side

SET B

- 1) Write a Python program to write a TCP server and TCP client with port number 50000.
Connect client to server
 - a) Accept a number from user at client side
 - b) Send the accepted number from client to server
 - c) At server side, check whether given number is prime or not.
 - d) Display output at server side
- 2) Write a Python program to write a TCP server and TCP client with port number 51000.
Connect client to server
 - a) Client should send message to server
 - b) Display message on server side
 - c) Continue this until client sends “quit” message

SET C

1) Write a Python program to write a TCP server and TCP client with port number 50000.

Connect client to server

- a) Accept a file name from user at client side
- b) Read the file at client side
- c) Send the file content from client to server
- d) Save file at server side as “received.txt”
- e) Display a message at server side “File received successfully”

Assignment Evaluation

0: Not done

2: Late Complete

4: Complete

1: Incomplete

3: Needs improvement

5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 7: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____

Assignment 8: Database Connectivity & Visualization

Database Connectivity

1) What is Database Connectivity?

Database connectivity refers to the ability of a programming language or application to connect to a database server and perform operations such as:

- Storing data
- Retrieving data
- Updating data
- Deleting data

In simple terms, **it is the communication bridge between a Python program and a database.**

2) Why is Database Connectivity Needed?

Database connectivity is essential because:

1. **Persistent Storage:**

Programs often need to store data beyond the program's execution.

2. **Large Data Handling:**

Databases can handle large datasets efficiently.

3. **Data Security and Consistency:**

Databases ensure safe and consistent storage of critical information.

4. **Real-World Applications:**

Most applications (banking, e-commerce, hospital systems, social media) rely on database connectivity.

3) Python Database API – DB-API 2.0

Python's standard specification for database connectivity is called **DB-API 2.0**, which defines:

- How Python connects to databases
- How to execute SQL queries
- How to handle transactions
- Standard objects like:
 - Connection

- Cursor

DB-API Structure

Component	Description
Connection Object	Establishes connection to database
Cursor Object	Executes SQL queries
Methods	execute(), fetchone(), fetchall()
Error Classes	DatabaseError, IntegrityError, etc.

4) PostgreSQL – Database Connectivity Using psycopg2

psycopg2 is the most widely used library for connecting Python to PostgreSQL.

a) Importing psycopg2

```
import psycopg2
```

b) Connecting to PostgreSQL

```
import psycopg2
conn = psycopg2.connect(
    dbname="studentdb",
    user="postgres",
    password="admin123",
    host="localhost",
    port="5432"
)
print("Connected to PostgreSQL!")
```

Important:

- This returns a **Connection object**.
- conn.cursor() gives a **Cursor object**.

c) Creating a Table (PostgreSQL)

```
cur = conn.cursor()
cur.execute("""
CREATE TABLE IF NOT EXISTS students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    marks INT
```

```

        )
    """
    conn.commit()
    print("Table created successfully!")

d) Insert Data (Create)
    cur.execute("INSERT INTO students (name, marks) VALUES (%s, %s)", ("Rahul",
85))
    conn.commit()

e) Read Data
    cur.execute("SELECT * FROM students")
    rows = cur.fetchall()
    for row in rows:
        print(row)

f) Update Data
    cur.execute("UPDATE students SET marks=%s WHERE id=%s", (90, 1))
    conn.commit()

g) Delete Data
    cur.execute("DELETE FROM students WHERE id=%s", (1,))
    conn.commit()

h) Closing Connection
    cur.close()
    conn.close()

```

5) SQLite – Database Connectivity Using sqlite3

SQLite is a **lightweight, file-based database**.

Python has built-in support using the sqlite3 module.

a) **Connecting to SQLite**

```

import sqlite3

conn = sqlite3.connect("school.db")

print("SQLite database connected!")

```

- If the file does not exist, Python **creates it automatically**.

b) **Creating a Table (SQLite)**

```

cur = conn.cursor()

```

```

cur.execute("""
CREATE TABLE IF NOT EXISTS student (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    marks INTEGER
)
""")
conn.commit()

```

c) CRUD Operations in SQLite

i) Insert Data

```

cur.execute("INSERT INTO student (name, marks) VALUES (?, ?)", ("Asha",
78))
conn.commit()

```

ii) Read Data

```

cur.execute("SELECT * FROM student")
data = cur.fetchall()
for row in data:
    print(row)

```

iii) Update Data

```

cur.execute("UPDATE student SET marks=? WHERE id=?", (88, 1))
conn.commit()

```

iv) Delete Data

```

cur.execute("DELETE FROM student WHERE id=?", (1,))
conn.commit()

```

6) Parameterized Queries

Parameterized queries help secure database operations.

i) PostgreSQL Example

```

cur.execute("INSERT INTO students (name, marks) VALUES (%s, %s)",
(username, score))

```

ii) SQLite Example

```

cur.execute("SELECT * FROM student WHERE name=?", (username,))

```

7) Error Handling in Database Operations

Use try-except to catch database errors.

Example

try:

```
conn = sqlite3.connect("data.db")
cur = conn.cursor()
cur.execute("INSERT INTO student VALUES (?, ?, ?)", (1, "Amit", 90))
conn.commit()
```

except sqlite3.DatabaseError as e:

```
print("Database error occurred:", e)
```

finally:

```
conn.close()
```

8) Summary of Important Functions

Function	Description
connect()	Connect to database
cursor()	Create cursor object
execute()	Execute SQL command
fetchone()	Fetch one row
fetchall()	Fetch all rows
commit()	Save changes
rollback()	Undo transaction
close()	Close connection

SET A

1) Write a Python program to:

- Connect to an SQLite database named college.db.
- Create a table students(id INTEGER PRIMARY KEY, name TEXT, marks INTEGER).
- Insert minimum 5 record into the table.

- d) Display a message *“Record inserted successfully”*.
- 2) Write a Python program to:
 - a) Connect to college.db.
 - b) Retrieve all records from students table.(Refer ‘students’ table which is created in Q1)
 - c) Display the records in a formatted manner.

SET B

- 1) Write a Python program to perform the following operations on an SQLite database. User ‘students’ table (from SET A Q1). Perform following operations:
 - a) Insert new student
 - b) Display all students
 - c) Update marks of a student. Accept student id from user as an input.
 - d) Delete a student record. Accept student id which you want to delete
 - e) ExitDo proper error handling (try-except).
- 2) Write a Python program using psycopg2 to:
 - a) Connect to a PostgreSQL database.
 - b) Create table employees(id SERIAL PRIMARY KEY, name VARCHAR(50), salary INT).
 - c) Ask user to input N employees, and insert their details using a parameterized query
 - d) Display the number of rows inserted.

SET C

- 1) Create a Python program using psycopg2 that performs the following:
 - a) Connect to database
 - b) Create a table products(pid SERIAL PRIMARY KEY, pname VARCHAR(50), price INT, qty INT).
 - c) Perform the following operations
 - i) Add new product
 - ii) Search product by name
 - iii) Update quantity of a product
 - iv) Delete a product
 - v) Show products with price > user input

- vi) Exit
- d) Handle following errors
 - i) invalid numeric input
 - ii) database connection issues
 - iii) record not found

Bonus: Implement validation for price and quantity (must be positive integers).

Use parameterized queries everywhere.

Assignment Evaluation

0: Not done 2: Late Complete 4: Complete
 1: Incomplete 3: Needs improvement 5: Well Done

Viva Questions:

Sr. No.	Questions Asked	Marks(1/0)
1		
2		
3		
4		
5		

Assignment 8: Total Marks _____ out of 10

Signature of the instructor

Date: ____/____/____